



Day-9

Python Set

Python Set

- **A Python set is the collection of the unordered items. Each element in the set must be unique, immutable, and the sets remove the duplicate elements. Sets are mutable which means we can modify it after its creation.**
- Unlike other collections in Python, there is no index attached to the elements of the set, i.e., we cannot directly access any element of the set by the index. However, we can print them all together, or we can get the list of elements by looping through the set.
- Creating a set
- The set can be created by enclosing the comma-separated immutable items with the curly braces {}. Python also provides the set() method, which can be used to create the set by the passed sequence.

Example 1: Using curly braces

- `Days = {"Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"}`
- `print(Days)`
- `print(type(Days))`
- `print("looping through the set elements ... ")`
- `for i in Days:`
 - `print(i)`
- Output:
 - `{'Friday', 'Tuesday', 'Monday', 'Saturday', 'Thursday', 'Sunday', 'Wednesday'}`
 - `<class 'set'>`
 - looping through the set elements ...
 - Friday
 - Tuesday
 - Monday
 - Saturday
 - Thursday
 - Sunday
 - Wednesday

Example 2: Using set() method

- `Days = set(["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"])`
- `print(Days)`
- `print(type(Days))`
- `print("looping through the set elements ... ")`
- `for i in Days:`
 - `print(i)`
- Output:
 - `{'Friday', 'Wednesday', 'Thursday', 'Saturday', 'Monday', 'Tuesday', 'Sunday'}`
 - `<class 'set'>`
 - looping through the set elements ...
 - Friday
 - Wednesday
 - Thursday
 - Saturday
 - Monday
 - Tuesday
 - Sunday

It can contain any type of element such as integer, float, tuple etc. But mutable elements (list, dictionary, set) can't be a member of set.

Consider the following example.

- # Creating a set which have immutable elements
- `set1 = {1,2,3, "Domain Computer", 20.5, 14}`
- `print(type(set1))`
- #Creating a set which have mutable element
- `set2 = {1,2,3,["DomainComputer",4]}`
- `print(type(set2))`

- Output:
- <class 'set'>
- Traceback (most recent call last)
- <ipython-input-5-9605bb6fbc68> in <module>
- 4
- 5 #Creating a set which holds mutable elements
- ----> 6 set2 = {1,2,3,["Domain Computer",4]}
- 7 print(type(set2))
- TypeError: unhashable type: 'list'

- In the above code, we have created two sets, the set set1 have immutable elements and set2 have one mutable element as a list. While checking the type of set2, it raised an error, which means set can contain only immutable elements.
- Creating an empty set is a bit different because empty curly {} braces are also used to create a dictionary as well. So Python provides the set() method used without an argument to create an empty set.
- # Empty curly braces will create dictionary
- set3 = {}
- print(type(set3))
-
- # Empty set using set() function
- set4 = set()
- print(type(set4))
- Output:
- <class 'dict'>
- <class 'set'>

Let's see what happened if we provide the duplicate element to the set.

- `set5 = {1,2,4,4,5,8,9,9,10}`
- `print("Return set with unique elements:",set5)`
- Output:
 - Return set with unique elements: {1, 2, 4, 5, 8, 9, 10}
 - In the above code, we can see that set5 consisted of multiple duplicate elements when we printed it remove the duplicity from the set.

Adding items to the set

- Python provides the add() method and update() method which can be used to add some particular item to the set. **The add() method is used to add a single element whereas the update() method is used to add multiple elements to the set.** Consider the following example.

- Example: 1 - Using add() method
- Months = set(["January", "February", "March", "April", "May", "June"])
- print("\nprinting the original set ... ")
- print(months)
- print("\nAdding other months to the set...");
- Months.add("July");
- Months.add ("August");
- print("\nPrinting the modified set...");
- print(Months)
- print("\nlooping through the set elements ... ")
- for i in Months:
- print(i)

Output:

- printing the original set ...
- {'February', 'May', 'April', 'March', 'June', 'January'}
- Adding other months to the set...
- Printing the modified set...
- {'February', 'July', 'May', 'April', 'March', 'August', 'June', 'January'}
- looping through the set elements
- ...
- February
- July
- May
- April
- March
- August
- June
- January

To add more than one item in the set, Python provides the `update()` method. It accepts iterable as an argument

- .
- Consider the following example.
- Example - 2 Using `update()` function
- `Months = set(["January", "February", "March", "April", "May", "June"])`
- `print("\nprinting the original set ... ")`
- `print(Months)`
- `print("\nupdating the original set ... ")`
- `Months.update(["July", "August", "September", "October"]);`
- `print("\nprinting the modified set ... ")`
- `print(Months);`

- Output:
- printing the original set ...
- {'January', 'February', 'April', 'May', 'June', 'March'}
- updating the original set ...
- printing the modified set ...
- {'January', 'February', 'April', 'August', 'October', 'May', 'June', 'July', 'September', 'March'}

Removing items from the set

- Python provides the discard() method and remove() method which can be used to remove the items from the set. The difference between these function, using discard() function if the item does not exist in the set then the set remain unchanged whereas remove() method will through an error.
- Consider the following example.

Example-1 Using discard() method

- months = set(["January","February", "March", "April", "May", "June"])
- print("\nprinting the original set ... ")
- print(months)
- print("\nRemoving some months from the set...");
- months.discard("January");
- months.discard("May");
- print("\nPrinting the modified set...");
- print(months)
- print("\nlooping through the set elements ... ")
- for i in months:
- print(i)

- Output:
- printing the original set ...
- {'February', 'January', 'March', 'April', 'June', 'May'}
- Removing some months from the set...
- Printing the modified set...
- {'February', 'March', 'April', 'June'}
- looping through the set elements ...
- February
- March
- April
- June

Python provides also the `remove()` method to remove the item from the set. Consider the following example to remove the items using `remove()` method.

- **Example-2 Using `remove()` function**
- `months = set(["January", "February", "March", "April", "May", "June"])`
- `print("\nprinting the original set ... ")`
- `print(months)`
- `print("\nRemoving some months from the set...");`
- `months.remove("January");`
- `months.remove("May");`
- `print("\nPrinting the modified set...");`
- `print(months)`

- Output:
- printing the original set ...
- {'February', 'June', 'April', 'May', 'January', 'March'}
- Removing some months from the set...
- Printing the modified set...
- {'February', 'June', 'April', 'March'}

- We can also use the pop() method to remove the item. Generally, the pop() method will always remove the last item but the set is unordered, we can't determine which element will be popped from set.
- Consider the following example to remove the item from the set using pop() method.
- Months = set(["January", "February", "March", "April", "May", "June"])
- print("\nprinting the original set ... ")
- print(Months)
- print("\nRemoving some months from the set...");
- Months.pop();
- Months.pop();
- print("\nPrinting the modified set...");
- print(Months)

- Output:
- printing the original set ...
- {'June', 'January', 'May', 'April', 'February', 'March'}
- Removing some months from the set...
- Printing the modified set...
- {'May', 'April', 'February', 'March'}

In the above code, the last element of the Month set is March but the pop() method removed the June and January because the set is unordered and the pop() method could not determine the last element of the set.

Python provides the `clear()` method to remove all the items from the set.

- Consider the following example.
- `Months = set(["January", "February", "March", "April", "May", "June"])`
- `print("\nprinting the original set ... ")`
- `print(Months)`
- `print("\nRemoving all the items from the set...");`
- `Months.clear()`
- `print("\nPrinting the modified set...")`
- `print(Months)`

- Output:
- printing the original set ...
- {'January', 'May', 'June', 'April', 'March', 'February'}
- Removing all the items from the set...
- Printing the modified set...
- set()

Difference between discard() and remove()

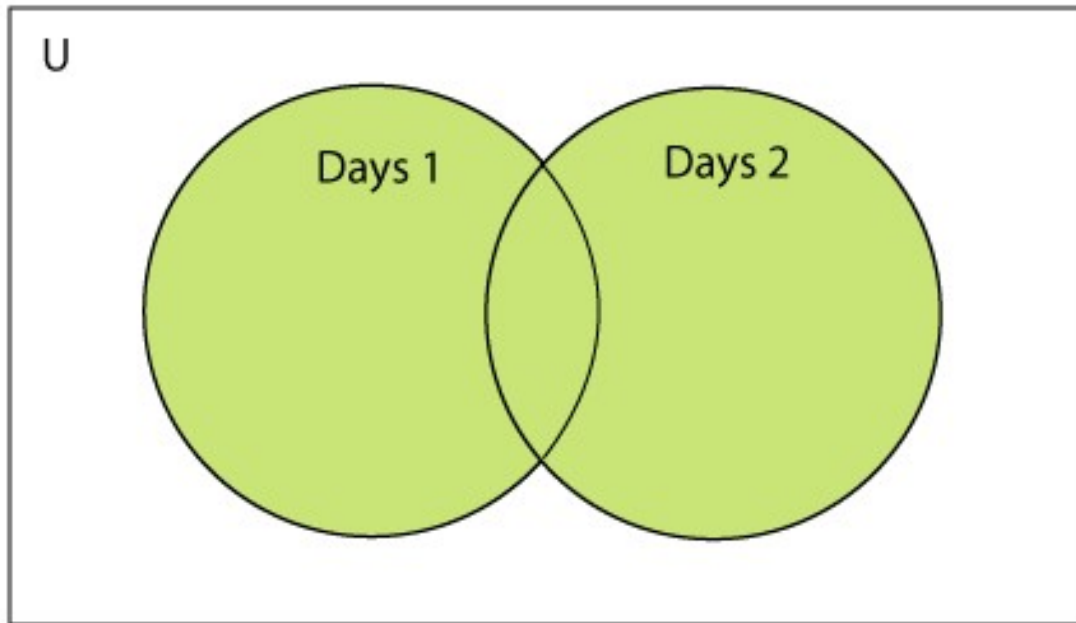
- Despite the fact that discard() and remove() method both perform the same task, There is one main difference between discard() and remove().
- If the key to be deleted from the set using discard() doesn't exist in the set, the Python will not give the error. The program maintains its control flow.
- On the other hand, if the item to be deleted from the set using remove() doesn't exist in the set, the Python will raise an error.
- Consider the following example.

- Example-
- Months = set(["January", "February", "March", "April", "May", "June"])
- print("\nprinting the original set ... ")
- print(Months)
- print("\nRemoving items through discard() method...");
- Months.discard("Feb"); #will not give an error although the key feb is not available in the set
- print("\nprinting the modified set...")
- print(Months)
- print("\nRemoving items through remove() method...");
- Months.remove("Jan") #will give an error as the key jan is not available in the set.
- print("\nPrinting the modified set...")
- print(Months)

- Output:
- printing the original set ...
- {'March', 'January', 'April', 'June', 'February', 'May'}
- Removing items through discard() method...
- printing the modified set...
- {'March', 'January', 'April', 'June', 'February', 'May'}
- Removing items through remove() method...
- Traceback (most recent call last):
- File "set.py", line 9, in
- Months.remove("Jan")
- KeyError: 'Jan'

Python Set Operations

- Set can be performed mathematical operation such as union, intersection, difference, and symmetric difference. Python provides the facility to carry out these operations with operators or methods. We describe these operations as follows.
- **Union of two Sets**
- **The union of two sets is calculated by using the pipe (|) operator.** The union of the two sets contains all the items that are present in both the sets.

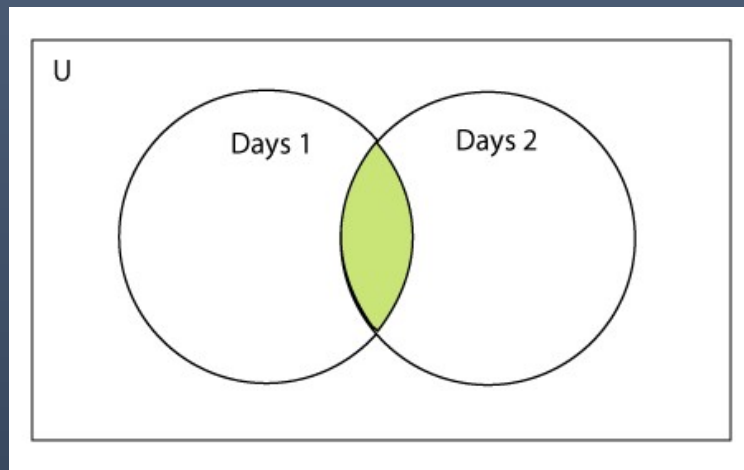


- Consider the following example to calculate the union of two sets.
- Example 1: using union | operator
- Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday", "Sunday"}
- Days2 = {"Friday", "Saturday", "Sunday"}
- print(Days1 | Days2) #printing the union of the sets
- Output:
- {'Friday', 'Sunday', 'Saturday', 'Tuesday', 'Wednesday', 'Monday', 'Thursday'}

- Python also provides the union() method which can also be used to calculate the union of two sets. Consider the following example.
- Example 2: using union() method
- Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday"}
- Days2 = {"Friday", "Saturday", "Sunday"}
- print(Days1.union(Days2)) #printing the union of the sets
- Output:
- {'Friday', 'Monday', 'Tuesday', 'Thursday', 'Wednesday', 'Sunday', 'Saturday'}

Intersection of two sets

- The intersection of two sets can be performed by the and & operator or the intersection() function. The intersection of the two sets is given as the set of the elements that common in both sets.



- Consider the following example.
- Example 1: Using & operator
- Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday"}
- Days2 = {"Monday", "Tuesday", "Sunday", "Friday"}
- print(Days1&Days2) #prints the intersection of the two sets
- Output:
- {'Monday', 'Tuesday'}

- Example 2: Using intersection() method
- `set1 = {"Devansh", "John", "David", "Martin"}`
- `set2 = {"Steve", "Milan", "David", "Martin"}`
- `print(set1.intersection(set2))` #prints the intersection of the two sets
- Output:
- `{'Martin', 'David'}`

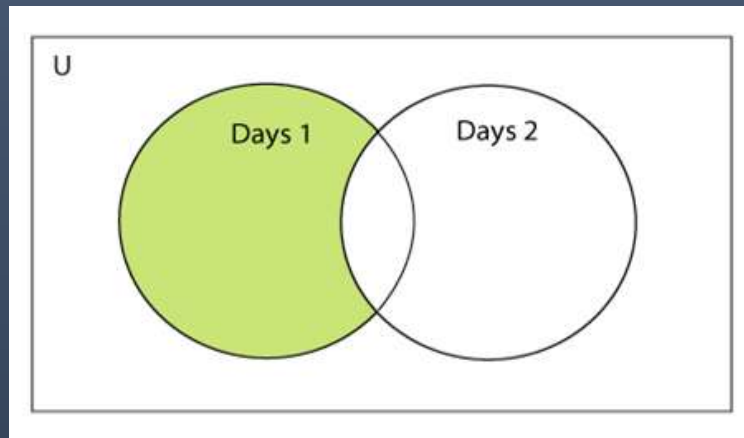
- Example 3:
- `set1 = {1,2,3,4,5,6,7}`
- `set2 = {1,2,20,32,5,9}`
- `set3 = set1.intersection(set2)`
- `print(set3)`
- Output:
- `{1,2,5}`

- The `intersection_update()` method
- The `intersection_update()` method removes the items from the original set that are not present in both the sets (all the sets if more than one are specified).
- The `intersection_update()` method is different from the `intersection()` method since it modifies the original set by removing the unwanted items, on the other hand, the `intersection()` method returns a new set.
- Consider the following example.
- `a = {"Devansh", "bob", "castle"}`
- `b = {"castle", "dude", "emyway"}`
- `c = {"fuson", "gaurav", "castle"}`
-
- `a.intersection_update(b, c)`
-
- `print(a)`

- Output:
- {'castle'}

Difference between the two sets

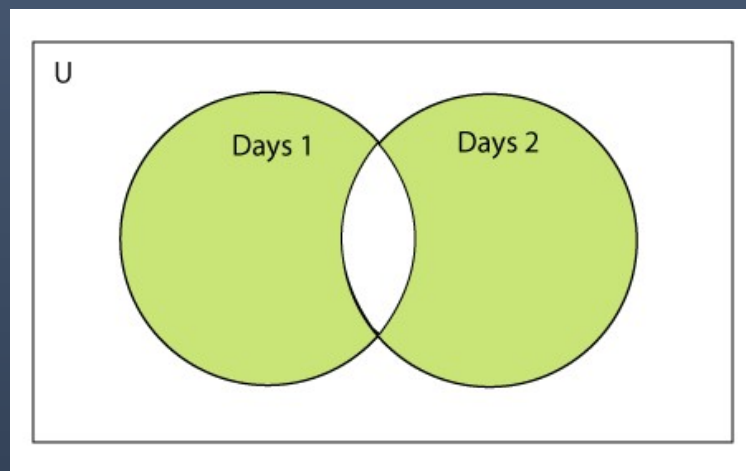
- **The difference of two sets can be calculated by using the subtraction (-) operator or intersection() method.** Suppose there are two sets A and B, and the difference is $A-B$ that denotes the resulting set will be obtained that element of A, which is not present in the set B.



- Consider the following example.
- Example 1 : Using subtraction (-) operator
- Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday"}
- Days2 = {"Monday", "Tuesday", "Sunday"}
- print(Days1-Days2) #{"Wednesday", "Thursday" will be printed}
- Output:
- {'Thursday', 'Wednesday'}

- Example 2 : Using difference() method
- Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday"}
- Days2 = {"Monday", "Tuesday", "Sunday"}
- print(Days1.difference(Days2)) # prints the difference of the two sets Days1 and Days2
- Output:
- {'Thursday', 'Wednesday'}

- Symmetric Difference of two sets
- **The symmetric difference of two sets is calculated by $\wedge$ operator or `symmetric_difference()` method. Symmetric difference of sets, it removes that element which is present in both sets. Consider the following example:**



- Example - 1: Using ^ operator

- $a = \{1, 2, 3, 4, 5, 6\}$

- $b = \{1, 2, 9, 8, 10\}$

- $c = a^b$

- `print(c)`

- Output:

- $\{3, 4, 5, 6, 8, 9, 10\}$

- Example - 2: Using `symmetric_difference()` method
- `a = {1,2,3,4,5,6}`
- `b = {1,2,9,8,10}`
- `c = a.symmetric_difference(b)`
- `print(c)`
- Output:
- `{3, 4, 5, 6, 8, 9, 10}`

Set comparisons

- Python allows us to use the comparison operators i.e., $<$, $>$, $<=$, $>=$, $=$ with the sets by using which we can check whether a set is a subset, superset, or equivalent to other set. The boolean true or false is returned depending upon the items present inside the sets.
- Consider the following example.

- Days1 = {"Monday", "Tuesday", "Wednesday", "Thursday"}
- Days2 = {"Monday", "Tuesday"}
- Days3 = {"Monday", "Tuesday", "Friday"}
-
- #Days1 is the superset of Days2 hence it will print true.
- print (Days1>Days2)
-
- #prints false since Days1 is not the subset of Days2
- print (Days1<Days2)
-
- #prints false since Days2 and Days3 are not equivalent
- print (Days2 == Days3)
- Output:
- True
- False
- False

Frozen Sets

- The frozen sets are the immutable form of the normal sets, i.e., the items of the frozen set cannot be changed and therefore it can be used as a key in the dictionary.
- The elements of the frozen set cannot be changed after the creation. We cannot change or append the content of the frozen sets by using the methods like `add()` or `remove()`.
- The `frozenset()` method is used to create the `frozenset` object. The iterable sequence is passed into this method which is converted into the frozen set as a return type of the method.
- Consider the following example to create the frozen set.

- `Frozenset = frozenset([1,2,3,4,5])`
- `print(type(Frozenset))`
- `print("\nprinting the content of frozen set...")`
- `for i in Frozenset:`
 - `print(i);`
- `Frozenset.add(6)` #gives an error since we cannot change the content of Frozenset after creation

- Output:
- `<class 'frozenset'>`
- printing the content of frozen set...
- 1
- 2
- 3
- 4
- 5
- Traceback (most recent call last):
- File "set.py", line 6, in `<module>`
- `Frozenset.add(6)` #gives an error since we can change the content of Frozenset after creation
- `AttributeError: 'frozenset' object has no attribute 'add'`

Frozenset for the dictionary

- If we pass the dictionary as the sequence inside the frozenset() method, it will take only the keys from the dictionary and returns a frozenset that contains the key of the dictionary as its elements.
- Consider the following example.
- Dictionary = {"Name":"John", "Country":"USA", "ID":101}
- print(type(Dictionary))
- Frozenset = frozenset(Dictionary); #Frozenset will contain the keys of the dictionary
- print(type(Frozenset))
- for i in Frozenset:
- print(i)

- Output:
- <class 'dict'>
- <class 'frozenset'>
- Name
- Country
- ID

Set Programming Example

- Example - 1: Write a program to remove the given number from the set.
- `my_set = {1,2,3,4,5,6,12,24}`
- `n = int(input("Enter the number you want to remove"))`
- `my_set.discard(n)`
- `print("After Removing:",my_set)`
- Output:
- Enter the number you want to remove:12
- After Removing: {1, 2, 3, 4, 5, 6, 24}

- Example - 2: Write a program to add multiple elements to the set.
- `set1 = set([1,2,4,"John","CS"])`
- `set1.update(["Apple","Mango","Grapes"])`
- `print(set1)`
- Output:
- `{1, 2, 4, 'Apple', 'John', 'CS', 'Mango', 'Grapes'}`

- Example - 3: Write a program to find the union between two set.
- `set1 = set(["Peter","Joseph", 65,59,96])`
- `set2 = set(["Peter",1,2,"Joseph"])`
- `set3 = set1.union(set2)`
- `print(set3)`
- Output:
- `{96, 65, 2, 'Joseph', 1, 'Peter', 59}`

- Example- 4: Write a program to find the intersection between two sets.
- `set1 = {23,44,56,67,90,45,"Domain"}`
- `set2 = {13,23,56,76,"Sachin"}`
- `set3 = set1.intersection(set2)`
- `print(set3)`
- Output:
- `{56, 23}`

- Example - 5: Write the program to add element to the frozenset.
- `set1 = {23,44,56,67,90,45,"Virat"}`
- `set2 = {13,23,56,76,"Sachin"}`
- `set3 = set1.intersection(set2)`
- `print(set3)`
- Output:
- `TypeError: 'frozenset' object does not support item assignment`

Above code raised an error because frozensets are immutable and can't be changed after creation.

- Example - 6: Write the program to find the issuperset, issubset and superset.
- `set1 = set(["Peter","James","Camroon","Ricky","Donald"])`
- `set2 = set(["Camroon","Washington","Peter"])`
- `set3 = set(["Peter"])`
-
- `issubset = set1 >= set2`
- `print(issubset)`
- `issuperset = set1 <= set2`
- `print(issuperset)`
- `issubset = set3 <= set2`
- `print(issubset)`
- `issuperset = set2 >= set3`
- `print(issuperset)`

- Output:

- False

- False

- True

- True

Python Built-in set methods

- Python contains the following methods to be used with the sets.

SN	Method	Description
1	<u>add(item)</u>	It adds an item to the set. It has no effect if the item is already present in the set.
2	<code>clear()</code>	It deletes all the items from the set.
3	<code>copy()</code>	It returns a shallow copy of the set.
4	<code>difference_update(...)</code>	It modifies this set by removing all the items that are also present in the specified sets.
5	<u>discard(item)</u>	It removes the specified item from the set.

6	intersection()	It returns a new set that contains only the common elements of both the sets. (all the sets if more than two are specified).
7	intersection_update(...)	It removes the items from the original set that are not present in both the sets (all the sets if more than one are specified).
8	Isdisjoint(...)	Return True if two sets have a null intersection.
9	Issubset(...)	Report whether another set contains this set.

10	Issuperset(...)	Report whether this set contains another set.
11	pop()	Remove and return an arbitrary set element that is the last element of the set. Raises KeyError if the set is empty.
12	remove(item)	Remove an element from a set; it must be a member. If the element is not a member, raise a KeyError.
13	symmetric_difference(...)	Remove an element from a set; it must be a member. If the element is not a member, raise a KeyError.
14	symmetric_difference_update(...)	Update a set with the symmetric difference of itself and another.
15	union(...)	Return the union of sets as a new set. (i.e. all elements that are in either set.)
16	update()	Update a set with the union of itself and others.

This PPT contents are

TO

© copyright
all rights reserved

Domain®
Computer Education
TRAINING & PLACEMENT CENTRE



TRAINING & PLACEMENT CENTRE

DOMAIN COMPUTER EDUCATION

2nd floor, Balaji Celebration, Ganpati
Mandir Road, Vishrambagh, Sangli

7620575756



DOMAIN COMPUTER EDUCATION

3rd Floor Prabhakar Plaza, Station road
Kolhapur.

Ph.No.- 9225804090



Website: www.domaincomputer.org